Python Functions and Parameters

Learning objective: Explain how to define and call Python functions, distinguish parameters from arguments, and demonstrate positional and keyword arguments.

Estimated duration: 45 minutes (limit 45)

Audience: Beginner Python programmers who know basic syntax (variables, print, indentation) but have not yet written their own functions · **Difficulty:** beginner

Overview

This lesson teaches you how to package reusable logic into Python functions using the `def` keyword, and how to supply data to those functions when you call them. You'll learn the precise difference between a parameter (the named slot in a function's definition) and an argument (the actual value you send in), and you'll practice both positional and keyword argument styles. By the end, you'll be able to write, call, and debug simple functions with confidence.

Learning objectives

- Define a Python function using the `def` keyword, correct syntax, and consistent indentation
- Call a Python function by name with the correct number of arguments, and explain what happens if you don't
- Distinguish a parameter (defined in the function signature) from an argument (passed at call time)
- Write and evaluate function calls that use positional arguments, where order determines matching
- Write and evaluate function calls that use keyword arguments, including mixing positional and keyword correctly

Why Functions Matter (4 min)

The problem functions solve

Imagine you need to greet 50 different users by name. Without functions, you'd copy-paste `print("Hello, " + name)` 50 times. A **function** lets you write that logic once and reuse it everywhere.

A Python function is a *self-contained block of code designed to perform a specific task*, which you can call and reuse in different parts of your program [S5]. Functions help organize code and make it more modular, reusable, and easier to maintain [S5].

Analogy: a function is a vending machine

Think of a function like a vending machine. You don't need to know how it makes coffee internally — you just press a button (call it) and provide coins (arguments). It does its work and gives you a result (the return value). The machine's **slots** for coins are like **parameters**; the actual **coins you insert** are like **arguments**.

By the end of this lesson you'll be able to build your own "vending machines" in Python.

Before You Start (3 min)

You should already be comfortable with:

- Writing and running basic Python statements (e.g., `print()`, variable assignment)
- Python's use of **indentation** to define blocks (no curly braces) [S8]
- Basic data types: strings, integers, lists

Tools needed

- Any Python 3 interpreter (python.org, an IDE, or an online REPL like the one at docs.python.org)

If you can run `print("hello")` and understand that indented lines belong to the same block as the `if` or `for` statement above them, you're ready.

Defining and Calling Functions (13 min)

Defining a function

To write a Python function you need a **header** that starts with the `def` keyword, followed by the function name, an optional comma-separated list of parameters inside a required pair of parentheses, and a final colon [S8]. The function's **body** is indented one level to the right and holds the code that runs when the function is called [S8].

```
def greet(name):
    print("Hello, " + name)
```

Here, `def` is the keyword for defining a function; the function name (`greet`) is followed by parameter(s) in parentheses; the colon signals the start of the function body, marked by indentation [S4].

Calling a function

You **call** a function by writing its name followed by parentheses, which enclose any necessary arguments [S5]:

```
greet("Maria")   # Output: Hello, Maria
```

The parentheses are *always required* in a call — even with zero arguments — because leaving them off just references the function object instead of running it [S8].

Parameters vs. arguments

This is a precise but often-confused pair of terms:

Term	Where it lives	Definition
Parameter	Function definition	The named variable listed inside the parentheses when the function is defined [S2]
Argument	Function call	The actual value sent to the function when it is called [S10]

```
def greet(name):      # 'name' is a PARAMETER
    print("Hello, " + name)

greet("Maria")         # "Maria" is an ARGUMENT
```

From the function's perspective: the parameter is the placeholder; the argument is what fills it in at run time [S6]. The two terms are often used loosely as synonyms, but being precise helps you read error messages correctly.

Matching counts: required arguments

By default, a function must be called with the exact number of arguments it expects [S10]. Calling `greet()` with zero arguments when `greet` requires one raises:

```
TypeError: greet() missing 1 required positional argument: 'name'
```

Calling `greet("A", "B")` with two arguments when `greet` only accepts one raises a different `TypeError`, because Python distinguishes missing arguments from extra arguments [S5][S7]:

```
TypeError: greet() takes 1 positional argument but 2 were given
```

Positional arguments

Positional arguments are matched to parameters strictly by their order in the call [S9][S5]. The first argument fills the first parameter, the second fills the second, and so on.

```
def describe_pet(animal, name):
    print(f"{name} is a {animal}")

describe_pet("dog", "Rex")   # animal='dog', name='Rex'
describe_pet("Rex", "dog")   # Swapped! animal='Rex', name='dog' — likely wrong
```

Keyword arguments

Keyword arguments are passed as `name=value` pairs and are matched by parameter name rather than position [S7][S2]. This removes the ordering restriction entirely — if you use keyword arguments, order does not matter [S9][S5]:

```
describe_pet(animal="dog", name="Rex")
describe_pet(name="Rex", animal="dog")   # Same result, different order
```

The ordering rule: positional before keyword

In a single call you *can* mix positional and keyword arguments, but **positional arguments must come before keyword arguments**, or Python raises a `SyntaxError` [S9][S7]:

```
describe_pet("dog", name="Rex")   # OK: positional first, then keyword
describe_pet(animal="dog", "Rex")  # SyntaxError: positional argument follows
keyword argument
```

Default parameter values

You can give a parameter a **default value** using `=` in the function signature, which makes that argument optional when calling [S9]:

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}")

greet("Sam")                 # Hello, Sam  (default used)
greet("Sam", "Hi")           # Hi, Sam     (default overridden)
```

Non-default parameters must come before default parameters in the definition; placing a default parameter first causes a `SyntaxError` [S9][S7].

Return values

The `return` statement ends function execution and sends a value back to the caller [S1][S8]. If no `return` statement is present, Python implicitly returns `None` [S8].

```
def add(a, b):
    return a + b

result = add(2, 3)   # result = 5
```

Function (local) scope

Variables created inside a function exist only in that function's **local scope** and are not accessible from outside it [S1]. Each call creates a fresh local symbol table [S1].

Docstrings

A **docstring** — a triple-quoted string as the very first statement in a function body — documents what the function does:

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

Worked Examples (8 min)

Example 1 — Define, call, and trace execution

```
def f(x):
    return x**2 + 1

print(f(4))   # 17
```

Step by step: `def f(x):` defines function `f` with parameter `x`. Calling `f(4)` passes argument `4`, binding `x = 4` inside the function. The body computes `4**2 + 1 = 17` and `return` sends `17` back to the caller [S4].

Example 2 — Positional order matters

```
def student(fn, ln='Mark', std='Fifth'):
    print(fn, ln, 'studies in', std, 'Standard')

student('John')                  # John Mark studies in Fifth Standard
student('John', 'Gates', 'Seventh')  # John Gates studies in Seventh Standard
student('John', 'Gates')         # John Gates studies in Fifth Standard
```

Notice how positional arguments fill in left to right, and any parameter not supplied falls back to its default [S9].

Example 3 — Keyword arguments in any order

```
def calculate_cost(item, quantity, price):
    print(f"{quantity} {item} cost ${quantity * price:.2f}")

calculate_cost(price=0.74, quantity=6, item="bananas")
# 6 bananas cost $4.44
```

Because each argument is named, Python unambiguously matches it to the correct parameter regardless of order [S5].

Example 4 — Common errors

```
def add_item(item_name, quantity):
    print(f"{quantity}x {item_name}")

add_item()
# TypeError: add_item() missing 2 required positional arguments: 'item_name' and
'quantity'

add_item(item_name="Bread", "3")
# SyntaxError: positional argument follows keyword argument
```

The first error occurs because required arguments were omitted [S7]; the second because a positional argument was placed after a keyword argument [S9].

Real-World Applications (2 min)

- **Data validation functions**: `def is_valid_email(address):` — called with one positional argument wherever you need to check input.
- **Configurable reports**: `def generate_report(data, title="Report", include_chart=False):` — keyword arguments let callers customize only the options they care about, leaving sensible defaults for the rest [S7].
- **APIs and libraries**: Many Python library functions (e.g., `print(*objects, sep=' ', end='\n')`) rely on keyword arguments so users don't have to memorize argument order for rarely-used options.

Practical Activity: Build and Call Your Own Function (7 min)

Task

1. Open a Python interpreter or file.
2. Define a function `book_ticket` with three parameters: `name`, `destination`, and `seat="Economy"` (default value). `python def book_ticket(name, destination, seat="Economy"): """Print a booking confirmation.""" print(f"{name} is booked to {destination} in {seat} class.")`
3. Call the function **three** different ways (one call per line, so you can compare outputs): - Using only positional arguments, including `seat`. - Using keyword arguments in a different order than the parameter list. - Mixing positional and keyword correctly (positional first), omitting `seat` so the default is used.
4. Deliberately write one call that puts a keyword argument before a positional one, run it, and read the error message.

Expected outcome

- Calls 1–3 print correct booking confirmations; call 2 shows the default `seat` value overridden.
- Call 4 raises `SyntaxError: positional argument follows keyword argument` — confirming the ordering rule [S9].

Self-check

- Can you explain, in your own words, why call 4 failed?
- Can you identify which lines are parameters and which are arguments in your code?

Knowledge check (6 min)

Answer all 6 questions below before checking the explanations.

Q1. Which keyword is used to begin a Python function definition, and what must immediately follow the function name?

Answer:

The `def` keyword; it must be followed by the function name, then a required pair of parentheses (optionally containing parameters), and a colon. — A function header starts with `def`, the function name, an optional parameter list inside required parentheses, and a final colon [S8].

Q2. What happens if you write `greet` (the function name) without parentheses instead of `greet()`?

- A. The function runs normally
- B. Python raises a SyntaxError
- C. You reference the function object but do not call/execute it
- D. Python returns None immediately

Answer:

C. You reference the function object but do not call/execute it — Parentheses are always required to actually call a function; without them, you are just referencing the function object, not invoking it [S8].

Q3. In `def add(a, b): return a + b` followed by `add(2, 3)`, which of these are the parameters and which are the arguments?

Answer:

`a` **and** `b` **are the parameters (named variables in the function definition);** `2` **and** `3` **are the arguments (actual values passed in the call).** — A parameter is the variable listed inside the parentheses in the function definition; an argument is the value sent to the function when called [S10][S2].

Q4. True or False: In the call `describe_pet("dog", "Rex")`, Python matches "dog" and "Rex" to parameters based on their names, not their order.

- True
- False

Answer:

False — These are positional arguments, which are matched to parameters strictly by order of appearance in the call, not by name [S9][S5].

Q5. Given `def calculate_cost(item, quantity, price):`, which call correctly uses keyword arguments?

- A. calculate_cost("bananas", 6, 0.74)
- B. calculate_cost(price=0.74, quantity=6, item="bananas")
- C. calculate_cost(item="bananas", 6, 0.74)
- D. calculate_cost(0.74, item="bananas", quantity=6)

Answer:

B. calculate_cost(price=0.74, quantity=6, item="bananas") — Keyword arguments use name=value pairs matched by parameter name, so they can appear in any order relative to each other [S5]. A uses only positional arguments. C is invalid because positional arguments (6, 0.74) follow a keyword argument, which is a SyntaxError. D is legal syntax (positional first), but 0.74 is assigned to `item` by position and then `item` is passed again by keyword, so Python raises `TypeError: calculate_cost() got multiple values for argument 'item'`.

Q6. Why does `describe_pet(animal="dog", "Rex")` raise a SyntaxError, and what rule explains it?

Answer:

Because a positional argument ("Rex") appears after a keyword argument (animal="dog"). The

rule is: positional arguments must always come before keyword arguments in a function call. — Python requires positional arguments to precede keyword arguments in a call; violating this order raises a SyntaxError [S9][S7].

Key Takeaways (2 min)

- Define functions with `def name(parameters):` followed by an indented body [S8].
- Call functions with `name(arguments)`; parentheses are always required [S8].
- **Parameters** live in the definition; **arguments** are the values passed at call time [S2][S10].
- **Positional arguments** match by order; **keyword arguments** match by name and can appear in any order relative to each other [S9][S5].
- Positional arguments must come before keyword arguments in a call, or you get a `SyntaxError` [S9].
- Default values (`param=value`) make arguments optional [S9]; missing required arguments raise `TypeError` [S7].
- `return` sends a value back to the caller; without it, a function returns `None` [S8].
- Variables defined inside a function are local to that function [S1].

Supplementary videos

- Functions in Python are easy — Good foundational overview of defining and calling functions before diving into argument types
- Python Arguments in Functions (Positional, Keywords & Default Arguments) #13 — Directly covers positional, keyword, and default arguments with examples matching this lesson
- Python keyword arguments are awesome! — Reinforces keyword argument syntax and why order-independence is useful

References

- [S1] 4. More Control Flow Tools — Official tutorial on argument passing rules, positional-only/keyword-only syntax, and scope
- [S2] Glossary — Precise official definitions of parameter and argument, including keyword vs. positional argument
- [S3] 8. Compound statements — Formal grammar and semantics of function definitions, defaults, and argument binding
- [S4] Python 3 Notes: User Defined Functions — Beginner-friendly walkthrough of def syntax and a worked example
- [S5] Defining Your Own Python Function – Real Python — Core explanations of defining/calling functions, positional vs keyword arguments, and error messages
- [S6] Parameter (computer programming) - Wikipedia — General definition and distinction between parameters and arguments across languages
- [S7] Using Python Optional Arguments When Defining Functions – Real Python — Covers default values, required vs optional parameters, and ordering errors
- [S8] The Python return Statement: Usage and Best Practices – Real Python — Explains explicit vs implicit return and function call basics
- [S9] Default arguments in Python - GeeksforGeeks — Worked examples of default arguments and argument ordering rules
- [S10] W3Schools.com — Simple parameter vs argument explanation and argument-count error example
- [S11] Automate the Boring Stuff with Python: Practical Programming for Total Beginners — Further reading: beginner-friendly book covering functions in practical projects

---

Research references - Python Functions and Parameters

Standards & documentation

- **[S1]** **4. More Control Flow Tools** — Python documentation
- Reliability: high (Official Python documentation (Python Software Foundation)); relevance 66%; status: verified — Page fetched (HTTP 200); 37,016 characters of text extracted.
- **[S2]** **Glossary** — Python documentation
- Reliability: high (Official Python documentation (Python Software Foundation)); relevance 58%; status: verified — Page fetched (HTTP 200); 67,351 characters of text extracted.
- **[S3]** **8. Compound statements** — Python documentation
- Reliability: high (Official Python documentation (Python Software Foundation)); relevance 43%; status: verified — Page fetched (HTTP 200); 61,801 characters of text extracted.

Websites & encyclopedias

- **[S4]** **Python 3 Notes: User Defined Functions** — sites.pitt.edu
- Reliability: high (Government or academic institution domain); relevance 42%; status: verified — Page fetched (HTTP 200); 5,371 characters of text extracted.
- **[S5]** **Defining Your Own Python Function – Real Python** — Real Python · Real Python
- Reliability: medium (Real Python - edited tutorial publisher); relevance 65%; status: verified — Page fetched (HTTP 200); 81,669 characters of text extracted.
- **[S6]** **Parameter (computer programming) - Wikipedia** — en.wikipedia.org
- Reliability: medium (Wikipedia - community-edited encyclopedia; good overview, verify against primary sources); relevance 58%; status: verified — Page fetched (HTTP 200); 23,230 characters of text extracted.
- **[S7]** **Using Python Optional Arguments When Defining Functions – Real Python** — Real Python · Real Python
- Reliability: medium (Real Python - edited tutorial publisher); relevance 57%; status: verified — Page fetched (HTTP 200); 41,405 characters of text extracted.
- **[S8]** **The Python return Statement: Usage and Best Practices – Real Python** — Real Python · Real Python
- Reliability: medium (Real Python - edited tutorial publisher); relevance 57%; status: verified — Page fetched (HTTP 200); 57,838 characters of text extracted.
- **[S9]** **Default arguments in Python - GeeksforGeeks** — GeeksforGeeks · 2018-08-16 00:50:37+00:00
- Reliability: medium (GeeksforGeeks - tutorial site; quality varies by article); relevance 51%; status: verified — Page fetched (HTTP 200); 5,448 characters of text extracted.
- **[S10]** **W3Schools.com** — W3Schools
- Reliability: medium (W3Schools - beginner tutorial site; simplified, verify details against specs); relevance 50%; status: verified — Page fetched (HTTP 200); 9,379 characters of text extracted.

Books (further reading - catalog records)

- **[S11]** **Automate the Boring Stuff with Python: Practical Programming for Total Beginners** — Al Sweigart · No Starch Press · 2015
- Reliability: medium (Open Library (Internet Archive) - library catalog record); relevance 57%; status: verified — Catalog record retrieved from openlibrary. Book content was not read; listed as further reading, not used as evidence.

Videos

- **[S12]** **Tutorial 16-Position And Keyword Argument In Functions|Python In Hindi|Krish Naik** — Krish Naik Hindi · 4y ago · 11:23
- Reliability: medium (YouTube video from Krish Naik Hindi; content not peer reviewed); relevance 45%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S13]** **Python Arguments in Functions (Positional, Keywords & Default Arguments) #13** — Programiz · 6y ago · 6:15

- Reliability: medium (YouTube video from Programiz; content not peer reviewed); relevance 36%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S14]** **Python keyword arguments are awesome!** — Bro Code · 3y ago · 6:44
- Reliability: high (YouTube - established educational channel (Bro Code)); relevance 27%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S15]** **Positional-only and keyword-only arguments in Python** — mCoding · 4y ago · 9:48
- Reliability: medium (YouTube video from mCoding; content not peer reviewed); relevance 20%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.
- **[S16]** **Functions in Python are easy** — Bro Code · 3y ago · 10:38
- Reliability: high (YouTube - established educational channel (Bro Code)); relevance 18%; status: verified — Existence, title and channel confirmed via YouTube oEmbed. Video content was not analysed.

Research notes

- Suggested URL could not be verified and was dropped: https://clouds.eos.ubc.ca/~phil/docs/problem_solving/07-Functions-and-Modules/07.07-Positional-and-Keyword-Arguments.html (ConnectError: [Errno 11001] getaddrinfo failed)

Research & Factual Accuracy Report - Python Functions and Parameters

Overall score: 82/100 · **Verdict:** Passed with recommendations · Lesson revision 5 · 2026-11-07T05:50:30+00:00 · claude_cli:sonnet

The lesson's core definitions and rules — def syntax, indentation, parameter vs. argument, positional vs. keyword argument matching, ordering rules, default values, return/None behavior, local scope, and docstrings — are all accurately and verbatim supported by the retrieved evidence. The main weaknesses are two illustrative TypeError messages (for a required-argument greet() call and for a too-many-arguments greet() call) that are factually standard Python behavior but are either uncited or cited to sources describing a different function/error condition, plus one uncited 'multiple values for argument' claim in the Q5 explanation.

Scoring methodology

Score = 0.5 x Claim support + 0.3 x Source quality + 0.2 x Citation coverage - Issue penalty. Claim support: share of extracted factual claims backed by evidence retrieved from verified sources (partially supported = half credit). Source quality: mean reliability of verified sources cited in the lesson (high 100 / medium 60 / low 25, rule-based by publisher). Citation coverage: share of teaching sections with at least one valid citation. Issue penalty: critical 15, high 8, medium 3, low 1 per open issue, capped at 40. Verdict: 'Needs revision' if any critical issue, any contradicted claim, or score < 70; 'Passed' if score >= 85 and no high-severity issue; otherwise 'Passed with recommendations'.

Component	Value	Weight	Detail
Claim support	94.0	0.5	23 supported, 1 partial, 1 unsupported, 0 contradicted of 25 claims
Source quality	73.3	0.3	9 verified sources cited
Citation coverage	100.0	0.2	4/4 teaching sections cite a source
Issue penalty	-7.0		3 issues

Verified claims (23)

- **C1** (supported) A Python function is a self-contained block of code designed to perform a specific task, which can be called and reused in different parts of a program. — *"A Python function is a*

self-contained block of code designed to perform a specific task, which you can call and reuse in different parts of your code." [S5]

- **C2** (supported) A function header starts with the def keyword, followed by the function name, an optional comma-separated parameter list inside required parentheses, and a final colon. — "*you need a header that starts with the def keyword, followed by the name of the function, an optional list of comma-separated arguments inside a required pair of parentheses, and a final colon.*" [S8] — Evidence calls the def-line items 'arguments'; 'parameters' is the more technically precise term for the definition context, which the lesson correctly uses.
- **C3** (supported) A function's body is indented one level to the right and holds the code that runs when the function is called. — "*the function's code block, which will begin one level of indentation to the right.*" [S8]
- **C4** (supported) A function is called by writing its name followed by parentheses enclosing any necessary arguments. — "*You call a Python function by writing its name followed by parentheses, enclosing any necessary arguments, to execute its code block.*" [S5]
- **C5** (supported) Parentheses are always required when calling a function, even with zero arguments; omitting them just references the function object instead of running it. — "*The parentheses, on the other hand, are always required in a function call. If you forget them, then you won't be calling the function but referencing it as a function object.*" [S8]
- **C6** (supported) A parameter is the named variable listed inside the parentheses when a function is defined. — "*A parameter is the variable listed inside the parentheses in the function definition.*" [S10]
- **C7** (supported) An argument is the actual value sent to a function when it is called. — "*An argument is the value that are sent to the function when it is called.*" [S10]
- **C8** (supported) By default, a function must be called with the exact number of arguments it expects. — "*By default, a function must be called with the correct number of arguments.*" [S10]
- **C11** (supported) Positional arguments are matched to parameters strictly by their order in the call. — "*For positional (non-keyword) arguments, order matters strictly.*" [S9]
- **C12** (supported) Keyword arguments are passed as name=value pairs and are matched by parameter name rather than position, so order does not matter when using them. — "*Keyword arguments allow assigning values by name, and order does not matter.*" [S9]
- **C13** (supported) In a single function call, positional arguments must come before keyword arguments, or Python raises a SyntaxError. — "*SyntaxError: positional argument follows keyword argument*" [S5]
- **C14** (supported) A parameter can be given a default value using = in the function signature, making that argument optional when calling. — "*If you assign a value to a given parameter in the definition of a function, then that value becomes a default argument value. Parameters with a default value are optional parameters*" [S5]
- **C15** (supported) Non-default parameters must come before default parameters in a function definition; placing a default parameter first causes a SyntaxError. — "*If you place a parameter with a default value before one without, then Python raises a SyntaxError. Required parameters must always come before optional ones.*" [S7]
- **C16** (supported) The return statement ends function execution and sends a value back to the caller. — "*An explicit return statement immediately ends the function's execution and sends the specified value back to the caller.*" [S8]
- **C17** (supported) If no return statement is present, Python implicitly returns None. — "*If no return statement is present, Python adds one implicitly, which returns None.*" [S8]
- **C18** (supported) Variables created inside a function exist only in that function's local scope and are not accessible from outside it; each call creates a fresh local symbol table. — "*When a function calls another function, or calls itself recursively, a new local symbol table is created for that call.*" [S1]
- **C19** (supported) A docstring is a triple-quoted string as the very first statement in a function body that documents what the function does. — "*The first statement of the function body can optionally be a string literal; this string literal is the function's documentation string, or docstring.*" [S1]
- **C20** (supported) In the worked example, calling f(4) where def f(x): return x**2 + 1 computes 4**2 + 1 = 17 and returns 17. — _">>> def f(x): return x**2 + 1

f(4) 17"_ [S4]

- **C21** (supported) In the student function example, calling student('John') with defaults ln='Mark', std='Fifth' prints 'John Mark studies in Fifth Standard'. — "*student('John') Output John Mark studies in Fifth Standard*" [S9]
- **C22** (supported) Calling calculate_cost(price=0.74, quantity=6, item="bananas") with all keyword arguments prints '6 bananas cost $4.44'. — "*calculate_cost(item="bananas", quantity=6, price=0.74) 6 bananas cost $4.44*" [S5] — Lesson order of keyword args differs from evidence example but keyword

matching makes order irrelevant, consistent with evidence.

- **C23** (supported) Calling add_item() when add_item requires item_name and quantity raises TypeError: add_item() missing 2 required positional arguments: 'item_name' and 'quantity'. — "*TypeError: add_item() missing 2 required positional arguments: 'item_name' and 'quantity'*" [S7]
- **C24** (supported) Calling add_item(item_name="Bread", "3") raises SyntaxError: positional argument follows keyword argument. — "*SyntaxError: positional argument follows keyword argument*" [S5] — General syntax rule confirmed; applies regardless of specific function name used in the lesson's example.
- **C25** (supported) Python uses indentation, not curly braces, to define blocks. — "*Python defines code blocks using indentation instead of brackets, begin and end keywords, and so on.*" [S8]

Partially supported claims (1)

- **C10** (partially_supported) Calling greet("A", "B") with two arguments when greet only accepts one raises TypeError: greet() takes 1 positional argument but 2 were given. — "*TypeError: function() takes 4 positional arguments but 5 positional arguments (and 1 keyword-only argument) were given*" [S5] — S5 confirms the general 'takes N positional arguments but M were given' error pattern for a different (4-parameter) function, supporting the format but not the exact wording for a 1-parameter greet() call. S7's evidence in this set discusses missing arguments, not extra arguments, so it does not support this specific claim.

Unsupported or contradicted claims (1)

- **C9** (unsupported) Calling greet() with zero arguments when greet requires one raises TypeError: greet() missing 1 required positional argument: 'name'. — "*greet(), no argument is passed, so Python uses the default value "Guest"*" [S9] — The retrieved evidence describes a greet(name='Guest') function WITH a default value, where calling greet() succeeds and prints 'Hello, Guest' rather than raising an error. No retrieved passage shows the exact TypeError text for a required (no-default) single-parameter function. The claim is standard, correct Python behavior, but the cited evidence does not confirm it.

Source quality assessment

Source	Reliability	Relevance	Status	Cited
[S1] 4. More Control Flow Tools	high	66%	verified	yes
[S2] Glossary	high	58%	verified	yes
[S3] 8. Compound statements	high	43%	verified	no
[S4] Python 3 Notes: User Defined Functions	high	42%	verified	yes
[S5] Defining Your Own Python Function – Real Python	medium	65%	verified	yes
[S6] Parameter (computer programming) - Wikipedia	medium	58%	verified	yes
[S7] Using Python Optional Arguments When Defining Functions – Real Python	medium	57%	verified	yes
[S8] The Python return Statement: Usage and Best Practices – Real Python	medium	57%	verified	yes

Source	Reliability	Relevance	Status	Cited
[S9] Default arguments in Python - GeeksforGeeks	medium	51%	verified	yes
[S10] W3Schools.com	medium	50%	verified	yes
[S11] Automate the Boring Stuff with Python: Practical Programming for Total Beginners	medium	57%	verified	no
[S12] Tutorial 16-Position And Keyword Argument In Functions	Python In Hindi	Krish Naik	medium	45%
[S13] Python Arguments in Functions (Positional, Keywords & Default Arguments) #13	medium	36%	verified	no
[S14] Python keyword arguments are awesome!	high	27%	verified	no
[S15] Positional-only and keyword-only arguments in Python	medium	20%	verified	no
[S16] Functions in Python are easy	high	18%	verified	no

Issues & recommended corrections

F1 · MEDIUM · missing_citation

- **Location:** Defining and Calling Functions > Matching counts: required arguments
- **Problem:** The exact error message 'TypeError: greet() missing 1 required positional argument: \'name\'' is presented as a factual Python output but has no citation attached to it, and none of the retrieved evidence for this claim shows this exact message for a required (no-default) single-parameter function.
- **Why it matters:** Learners rely on exact error-message wording to recognize errors in real code; presenting it without sourcing makes it unverifiable even though it happens to be correct.
- **Evidence:** Retrieved evidence instead shows a greet(name='Guest') function WITH a default value, where greet() succeeds and prints 'Hello, Guest' rather than erroring. (S9)
- **Original:** Calling `greet()` with zero arguments when `greet` requires one raises:

```
TypeError: greet() missing 1 required positional argument: 'name'
```

- **Suggested correction:** Calling `greet()` with zero arguments when `greet` requires one raises [S7]:

```
TypeError: greet() missing 1 required positional argument: 'name'
```

F2 · MEDIUM · wrong_citation

- **Location:** Defining and Calling Functions > Matching counts: required arguments
- **Problem:** The sentence citing [S5][S7] for the 'extra arguments' TypeError message is only partially supported: S5's retrieved passage shows this error pattern for a 4-parameter function (not greet), and S7's retrieved passage is about MISSING required arguments, not extra/too-many arguments, so S7 does not actually support this specific claim.
- **Why it matters:** Citing a source that addresses a different error condition (missing vs. extra arguments) can mislead learners into thinking that source directly documents this exact scenario, undermining trust in the citations.
- **Evidence:** S5: 'TypeError: function() takes 4 positional arguments but 5 positional arguments (and 1 keyword-only argument) were given'; S7's retrieved content is about 'add_item() missing 2 required positional arguments', not extra-argument errors. (S5, S7)
- **Original:** Calling `greet("A", "B")` with two arguments when `greet` only accepts one raises a different `TypeError`, because Python distinguishes missing arguments from extra arguments [S5][S7]:
- **Suggested correction:** Calling `greet("A", "B")` with two arguments when `greet` only accepts one raises a different `TypeError`, because Python distinguishes missing arguments from extra arguments [S5]:

F3 · LOW · missing_citation

- **Location:** Knowledge check > Q5 explanation (option D)
- **Problem:** The explanation for option D introduces a specific, uncited technical claim: 'Python raises TypeError: calculate_cost() got multiple values for argument \'item\''. No source is cited and none of the retrieved evidence for this lesson covers this exact error condition (positional argument duplicated by a later keyword argument of the same name).
- **Why it matters:** This is a precise technical detail that a learner could misremember or that could be wrong for a different Python version; leaving it unsourced reduces the lesson's verifiability even though the behavior described is standard CPython behavior.
- **Evidence:** No retrieved passage shows this 'got multiple values for argument' message.
- **Original:** D is legal syntax (positional first), but 0.74 is assigned to `item` by position and then `item` is passed again by keyword, so Python raises `TypeError: calculate_cost() got multiple values for argument 'item'`.
- **Suggested correction:** D is legal syntax (positional first), but 0.74 is assigned to `item` by position and then `item` is passed again by keyword, so Python raises a `TypeError` for a duplicate argument value — a case not covered by this lesson's cited sources, but consistent with Python's documented rule that each parameter can receive only one value [S9].

Research gaps

- No retrieved evidence confirms the exact TypeError wording for calling a required (no-default) single-parameter function with zero arguments (e.g., 'greet() missing 1 required positional argument'); the closest evidence (S7) demonstrates this pattern only for a two-parameter function.
- No retrieved evidence confirms the exact TypeError wording for calling a one-parameter function with two arguments ('takes 1 positional argument but 2 were given'); the closest evidence (S5) demonstrates the pattern only for a four-parameter function.
- No retrieved evidence addresses the 'got multiple values for argument' TypeError referenced in the Q5 explanation (duplicate assignment via positional + keyword argument of the same name).

Final verdict

Passed with recommendations (82/100)

Instructional Quality Report - Python Functions and Parameters

Overall score: 76/100 · **Verdict:** Passed with recommendations · Lesson revision 5 · 2026-11-07T05:47:09+00:00 · claude_cli:sonnet

This lesson is well-aligned to its objective, with clear explanations, strong analogies, relevant examples, and a knowledge check that maps directly onto each learning-objective part. Its main weakness is pacing:

the concepts section folds in several non-objective topics (return, scope, docstrings) that crowd out time for the core positional/keyword material, and both the activity and knowledge check pack more cognitive work than their minute allotments realistically support, with the activity's final step also risking a confusing script-wide failure for beginners.

Scoring methodology

Score = sum of ten rubric criteria scored 0-10 by an independent instructional-design review: Learning objective alignment, Explanation clarity, Logical sequence, Concept completeness, Quality of examples, Practical applicability, Student engagement, Knowledge-check relevance, Difficulty progression, 45-minute time compliance. The time-compliance criterion is forced to 0 by code when section minutes exceed 45. Deterministic checks add issues for missing required sections, fewer than 4 knowledge checks, or objective parts with no knowledge check. Verdict: 'Needs revision' if any critical issue, any objective part not covered, the 45-minute limit is exceeded, or score < 70; 'Passed' if score >= 85 and no high-severity issue; otherwise 'Passed with recommendations'.

Rubric

Criterion	Score	Notes
Learning objective alignment	9/10	All five objective parts (def keyword, calling, parameter/argument distinction, positional args, keyword args) are explicitly taught in the concepts section, reinforced in worked examples, practiced in the activity, and directly tested in the knowledge check. Alignment is tight and traceable end-to-end.
Explanation clarity	8/10	The vending-machine analogy, the parameter-vs-argument comparison table, and the explicit TypeError/SyntaxError examples are unusually clear for a beginner audience. One clarity gap: the lesson never explains that a SyntaxError is a parse-time failure (before any code runs) while a TypeError is a run-time failure, which matters for how the activity's error-producing call behaves relative to the other calls.
Logical sequence	8/10	Def → call → parameter/argument distinction → required-argument errors → positional → keyword → ordering rule → defaults → return → scope → docstrings is a sensible progression for the core objective, but return/scope/docstrings are inserted as if core material even though they sit outside the stated objective, adding length without a matching time allowance.

Criterion	Score	Notes
Concept completeness	9/10	Every objective part is covered in depth, and the lesson even goes beyond it (defaults, return values, scope, docstrings), which is generous for completeness but creates downstream timing pressure (see timing criterion).
Quality of examples	8/10	Four worked examples progress from a simple trace, to positional defaults, to pure keyword matching, to two distinct error types — strong variety and realistic patterns (student records, cost calculator). The error-timing nuance between the TypeError and SyntaxError examples in Example 4 is not explained, which is a minor clarity gap for examples otherwise well-chosen.
Practical applicability	7/10	Real-World Applications gives plausible use cases (validation, configurable reports, print()'s signature) but stays at the bullet-point level with no runnable code, so at only 2 minutes it reads more like a list than an application. The activity itself (ticket booking) is a good practical scenario.
Student engagement	6/10	The 13-minute concepts section is a long, dense run of prose + code blocks with no embedded checkpoint or 'predict the output' prompt until the learner reaches the separate Examples section; engagement relies almost entirely on the activity and knowledge check to break up passive reading.
Knowledge-check relevance	9/10	Each question maps to a distinct objective part (Q1 def, Q2 call semantics, Q3 parameter/argument, Q4 positional, Q5 keyword, Q6 ordering rule). Q5's distractors are especially strong (D tests a subtle 'multiple values for argument' case). Mixing 3 open-response and 3 MC items is pedagogically fine but adds time risk (see timing).

Criterion	Score	Notes
Difficulty progression	8/10	Progression from single-parameter greet() to multi-parameter describe_pet() to defaults in student() to all-keyword calculate_cost() to intentional error cases is well-graded; the activity mirrors this by building up to a deliberately broken call.
45-minute time compliance	4/10	Arithmetic sums to 45, but per-section load is unrealistic for true beginners: 13 minutes for ~11 subtopics (def, call, params vs args, required-argument errors, positional, keyword, ordering, defaults, return, scope, docstrings) is dense reading alone, let alone comprehension; the 7-minute activity asks a beginner to write a function, make 4 calls, run two different error types, and self-reflect; the 6-minute knowledge check mixes 3 constructed-response items with 3 MC items. Each of these three sections likely needs materially more time than allotted, or the extraneous content (return/scope/docstrings) needs to be trimmed to make room.

Objective alignment & coverage

Objective part	Coverage	Where	Notes
Explain how to define a Python function using the def keyword	full	Defining and Calling Functions (concepts)	Header syntax, parentheses, colon, and indentation are explained with a worked greet() example and tested in Q1.
Explain how to call a Python function	full	Defining and Calling Functions (concepts); Knowledge check	Covers call syntax, the required-parentheses rule, and the object-reference-vs-call distinction; tested in Q2.
Distinguish parameters from arguments	full	Defining and Calling Functions (concepts); Worked Examples; Knowledge check	Explicit comparison table plus repeated callouts in examples; tested in Q3 and the activity self-check.

Objective part	Coverage	Where	Notes
Demonstrate the use of positional arguments in a full function call		Defining and Calling Functions (concepts); Worked Examples (Ex. 1-2); Practical Activity; Knowledge check	Taught, exemplified (describe_pet, student()), practiced in the activity, and tested in Q4.
Demonstrate the use of keyword arguments in a full function call		Defining and Calling Functions (concepts); Worked Examples (Ex. 3-4); Practical Activity; Knowledge check	Taught with ordering rule, exemplified (calculate_cost), practiced in the activity, and tested in Q5 and Q6.

Clarity assessment

Explanations are generally excellent for the target audience, using concrete analogies (vending machine), a direct comparison table for parameters vs. arguments, and side-by-side code for each new idea. The one recurring clarity gap is around error semantics: the lesson shows both a TypeError (runtime) and a SyntaxError (parse-time) as if they are equivalent failure modes encountered 'when you run the call,' without ever stating that a SyntaxError prevents the whole file from executing at all — a distinction that matters directly for the activity's final step.

Example quality

The four worked examples are well-chosen and appropriately varied: a basic trace, a multi-default positional case, an all-keyword case, and two contrasting error cases. They reinforce the objective directly and use readable, realistic function names. The only weakness is that Example 4 juxtaposes a TypeError and a SyntaxError without clarifying that the second line would never even be reached if both lines were run together in one script, which could plant a misconception about how/when Python errors surface.

Activity quality

The ticket-booking activity is a strong, realistic task that requires the learner to produce all three argument-passing styles plus a deliberate failure case, which is pedagogically sound. However, as written, placing the SyntaxError-triggering call 4 in the same file as calls 1–3 (per the instructions) means the file would fail to parse before any of calls 1–3 could run, contradicting the stated 'Expected outcome' that calls 1–3 print correctly. This is a real risk of confusing beginners who won't understand why nothing printed. The task should instruct learners to test the broken call separately (comment it out or add/remove it after confirming the first three calls work).

Assessment quality

The six knowledge-check questions map cleanly onto each part of the objective and include some genuinely strong distractors (especially Q5's option D, which tests a subtle 'multiple values for argument' TypeError rather than just syntax). Explanations are thorough and reference-backed. The main risk is cognitive-load/time: three of six items (Q1, Q3, Q6) are open constructed-response rather than MC, which typically take longer to compose and self-grade than the 1-minute-per-question pace the 6-minute allotment implies.

Timing analysis

Total **45 min** of 45 (compliant). Arithmetic sums to 45, but per-section load is unrealistic for true beginners: 13 minutes for ~11 subtopics (def, call, params vs args, required-argument errors, positional, keyword, ordering, defaults, return, scope, docstrings) is dense reading alone, let alone comprehension; the 7-minute activity asks a beginner to write a function, make 4 calls, run two different error types, and self-reflect; the 6-minute knowledge check mixes 3 constructed-response items with 3 MC items. Each of these three sections likely needs materially more time than allotted, or the extraneous content (return/scope/docstrings) needs to be trimmed to make room.

- Why Functions Matter: 4 min
- Before You Start: 3 min
- Defining and Calling Functions: 13 min
- Worked Examples: 8 min
- Real-World Applications: 2 min
- Practical Activity: Build and Call Your Own Function: 7 min

- Knowledge check: 6 min
- Key Takeaways: 2 min

Missing concepts

- The distinction between when a SyntaxError occurs (before the program runs, during parsing) versus when a TypeError occurs (at call time) is used implicitly in Example 4 and the activity but never explicitly taught, even though it directly affects how the activity's final step should be carried out.

Suggested restructuring

- Condense the 'Return values', 'Function (local) scope', and 'Docstrings' subsections (none of which are part of the stated objective) into a brief 'Beyond this lesson' callout, freeing several minutes inside the 13-minute concepts block for deeper engagement with positional/keyword mechanics.
- Reallocate the reclaimed time toward the Practical Activity (currently 7 min for a 4-part, two-error-type task) and the Knowledge check (currently 6 min for 3 open-response + 3 MC items), both of which are tight for the target audience.
- Insert one short embedded 'predict the output' micro-check partway through the 13-minute concepts section (e.g., right after the Positional arguments subsection) to break up the long passive-reading stretch before the Worked Examples section.
- Revise the activity's step 4 so the SyntaxError-producing call is tested in isolation (after confirming calls 1–3 work), since leaving it in the same script as calls 1–3 would prevent the whole file from parsing and running.

Issues & recommended improvements

I1 · HIGH · timing

- **Location:** Defining and Calling Functions — 'Return values', 'Function (local) scope', and 'Docstrings' subsections
- **Problem:** The 13-minute concepts section packs in roughly eleven distinct subtopics (def syntax, calling, parameter/argument distinction, required-argument errors, positional, keyword, ordering rule, defaults, return values, scope, docstrings) for an audience that has never written a function before. Three of these (return, scope, docstrings) are outside the stated learning objective entirely.
- **Why it matters:** Beginners need processing time for each new idea; cramming non-objective content into an already-dense section risks shallow understanding of the actually-tested concepts (positional/keyword arguments) and makes the 13-minute budget unrealistic.
- **Original:** ### Return values The `return` statement ends function execution and sends a value back to the caller [S1][S8]. If no `return` statement is present, Python implicitly returns `None` [S8].

```
def add(a, b):
    return a + b

result = add(2, 3)   # result = 5
```

Function (local) scope

Variables created inside a function exist only in that function's **local scope** and are not accessible from outside it [S1]. Each call creates a fresh local symbol table [S1].

Docstrings

A **docstring** — a triple-quoted string as the very first statement in a function body — documents what the function does:

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

- **Suggested correction:** ### Beyond this lesson Two quick facts for later: a function can send a value back to its caller with `return` (and returns `None` automatically if you never use `return`), and variables created inside a function only exist inside that function (its **local scope**) — they're not visible outside it. You can also document a function with a **docstring**, a triple-quoted string right after `def`:

```
def add(a, b):
    """Return the sum of a and b."""
```

```
    return a + b
```

We'll dig into `return` and scope properly in a later lesson — for now, keep your focus on defining, calling, and passing positional/keyword arguments.

I2 · HIGH · activity

- **Location:** Practical Activity, Task step 4
- **Problem:** Step 4 tells the learner to write the keyword-before-positional call as another line in the same file as calls 1–3, then 'run it.' In Python, a SyntaxError is raised when the file is parsed, before any line executes — so if call 4 stays in the same file, calls 1–3 will never print anything either, contradicting the stated expected outcome.
- **Why it matters:** A beginner running the whole file and seeing no output at all (not even the first three correct confirmations) will likely be confused about why 'nothing worked,' undermining the very outcome the activity is trying to produce.
- **Original:** 4. Deliberately write one call that puts a keyword argument before a positional one, run it, and read the error message.
- **Suggested correction:** 4. First run the file with only calls 1–3 and confirm all three booking confirmations print correctly. Then, on a new line below them, add a call that puts a keyword argument before a positional one (e.g., `book_ticket(name="Ana", "Paris")`), save, and run the file again just to see the error message — then comment that line back out before moving on.

I3 · MEDIUM · activity

- **Location:** Practical Activity, Expected outcome
- **Problem:** The expected-outcome bullets state calls 1–3 print correctly and call 4 raises a SyntaxError, but don't clarify that a SyntaxError anywhere in a Python file blocks the entire file from running — so if call 4 is left in place alongside calls 1–3, the learner would see only the error and no prints at all, not both outcomes simultaneously.
- **Why it matters:** Without this clarification, learners may think they did something wrong when calls 1–3 don't print once call 4 is added, rather than understanding this as expected Python behavior.
- **Original:** - Call 4 raises `SyntaxError: positional argument follows keyword argument` — confirming the ordering rule [S9].
- **Suggested correction:** - Call 4 raises `SyntaxError: positional argument follows keyword argument` — confirming the ordering rule [S9]. Because Python detects this error while reading the file, before running anything, test call 4 by itself (temporarily removing or commenting out calls 1–3, or adding call 4 only after you've already seen calls 1–3 succeed) rather than expecting both outcomes from a single run.

I4 · MEDIUM · clarity

- **Location:** Worked Examples, Example 4 — Common errors
- **Problem:** The explanation names why each error occurs but never distinguishes that the TypeError happens at call/run time while the SyntaxError happens at parse time (before the program runs at all), even though both are presented back-to-back as if encountered the same way.
- **Why it matters:** Without this distinction, learners can't correctly predict what happens when a SyntaxError-causing line sits in a larger script (e.g., in the Practical Activity), and may misunderstand Python's error model more broadly.
- **Original:** The first error occurs because required arguments were omitted [S7]; the second because a positional argument was placed after a keyword argument [S9].
- **Suggested correction:** The first error occurs because required arguments were omitted [S7] — this is a `TypeError`, detected when Python actually tries to run the call. The second occurs because a positional argument was placed after a keyword argument [S9] — this is a `SyntaxError`, detected while Python reads the file, *before* any code runs. That means a single bad `SyntaxError` line can stop an entire script from running at all, not just the one call.

I5 · MEDIUM · timing

- **Location:** Knowledge check, overall question mix
- **Problem:** Six minutes is allotted for 6 questions, but 3 of them (Q1, Q3, Q6) are open constructed-response items requiring the learner to compose a written explanation, not just select an answer — this takes meaningfully longer per item than a multiple-choice question.

- **Why it matters:** At a true 1-minute-per-question pace, learners won't have time to actually think through and write out explanations for the open-response items, turning the check into a skim-the-answer exercise rather than genuine retrieval practice.

I6 · LOW · engagement

- **Location:** End of 'Positional arguments' subsection
- **Problem:** The 13-minute concepts section has no embedded interactive checkpoint between the introduction of positional arguments and the start of the separate Worked Examples section, making it a long passive read.
- **Why it matters:** Without a mid-section checkpoint, learners may lose focus or fail to self-correct a misunderstanding until much later, reducing retention of the positional-argument rule that the rest of the lesson builds on.
- **Suggested correction: Quick check:** Before reading on, predict the output of `describe_pet("Rex", "dog")` from the example above, then compare it to the comment shown. If your prediction was wrong, re-read the positional-matching rule before moving on.

Final verdict

Passed with recommendations (76/100)